

PRACTICE EXERCISES OF THE MICROPROCESSORS & MICROCONTROLLERS

Instructor: The Tung Than

Student's name: Nguyễn Đông Quân

Student code: 24521438

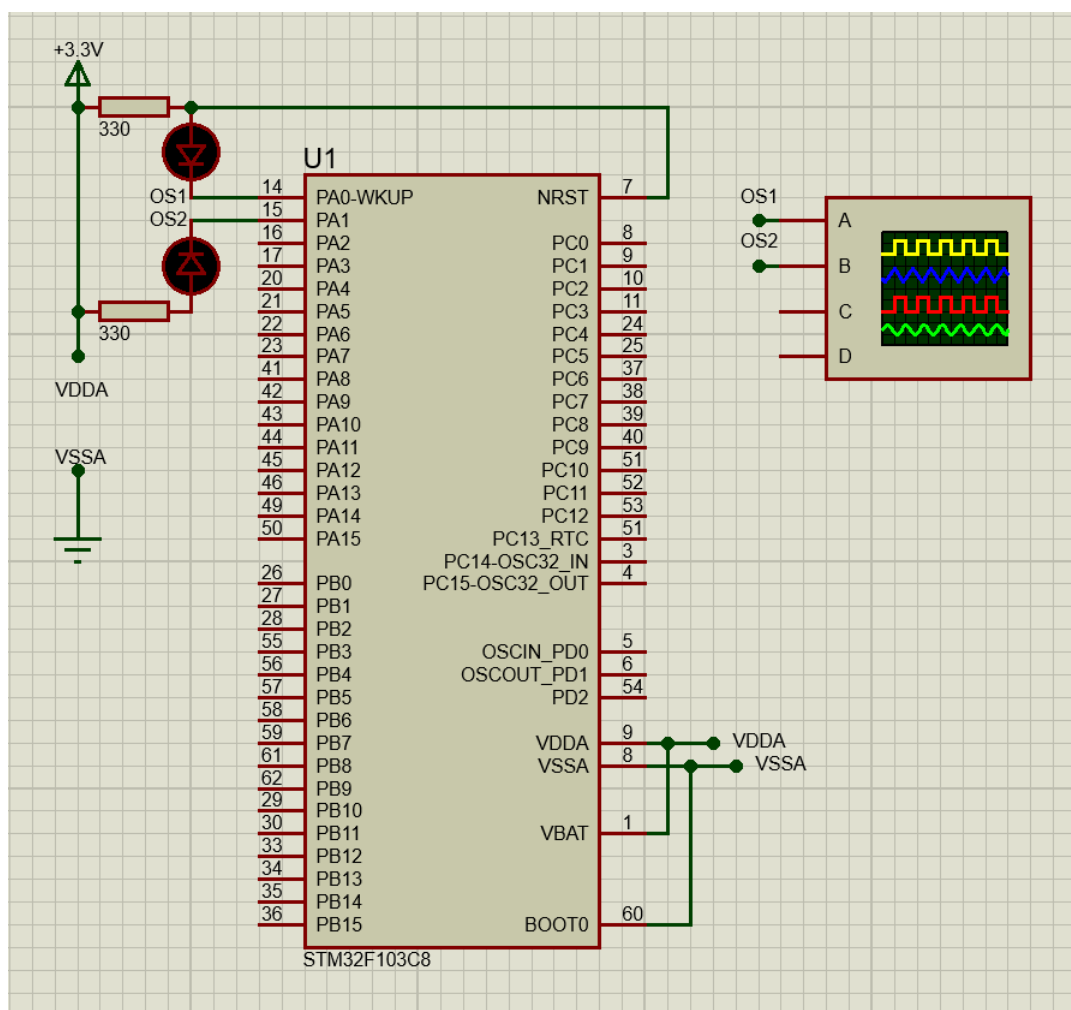
PRACTICE REPORT NO #5:

Tìm hiểu và thực hành về GPIO và Timer trên STM32F103C8

I. Exercise 1

Thiết kế mạch mô phỏng gồm vi điều khiển STM32F103C8 và hai LED đơn (LED1 và LED2). Yêu cầu:

- LED1 nhấp nháy với chu kỳ 100 ms.
- LED2 nhấp nháy với chu kỳ 1 giây.
- Sử dụng Timer để tạo độ trễ chính xác.



Mạch nhấp LED theo chu kỳ

Nguyên lý hoạt động của hệ thống:

Vì điều khiển STM32F103C8 điều khiển hai LED nối tại PA0 và PA1. TIM2 được cấu hình tạo ngắt định kỳ chính xác mỗi 50 ms bằng Timer. Mỗi lần ngắt xảy ra, LED1 được đảo trạng thái, nên chu kỳ nhấp nháy là 100 ms. LED2 chỉ đảo trạng thái sau 10 lần ngắt, tức mỗi 500 ms, nên chu kỳ nhấp nháy là 1 giây.

Giải thích kết quả mô phỏng:

```
#define RCC_CR (*(volatile unsigned int*)0x40021000)
#define RCC_CFGR (*(volatile unsigned int*)0x40021004)
#define RCC_APB2ENR (*(volatile unsigned int*)0x40021018)
#define RCC_APB1ENR (*(volatile unsigned int*)0x4002101C)

#define GPIOA_CRL (*(volatile unsigned int*)0x40010800)
#define GPIOA_ODR (*(volatile unsigned int*)0x4001080C)

#define TIM2_CR1 (*(volatile unsigned int*)0x40000000)
#define TIM2_DIER (*(volatile unsigned int*)0x4000000C)
#define TIM2_SR (*(volatile unsigned int*)0x40000010)
#define TIM2_EGR (*(volatile unsigned int*)0x40000014)
#define TIM2_CNT (*(volatile unsigned int*)0x40000024)
#define TIM2_PSC (*(volatile unsigned int*)0x40000028)
#define TIM2_ARR (*(volatile unsigned int*)0x4000002C)

#define NVIC_ISER0 (*(volatile unsigned int*)0xE000E100)

#define LED1 (1 << 0) // PA0
#define LED2 (1 << 1) // PA1

volatile unsigned int count_led2 = 0;

void TIM2_IRQHandler(void) // Every 50ms
{
    if (TIM2_SR & 1) // check UIF
    {
        TIM2_SR &= ~1; // clear UIF

        GPIOA_ODR ^= LED1; // LED1 toggle 50ms

        count_led2++;
```

```

        if (count_led2 >= 10) // 10 x 50ms = 500ms
        {
            count_led2 = 0;
            GPIOA_ODR ^= LED2; // LED2 toggle 500ms
        }
    }
}

int main(void)
{
    // HSI 8MHz
    RCC_CR |= 1; // HSI ON
    while (!(RCC_CR & (1 << 1))); // wait HSIRDY

    RCC_CFGR &= ~(3 << 0); // SYSCLK = HSI

    // Enable GPIOA & TIM2
    RCC_APB2ENR |= (1 << 2); // IOPAEN
    RCC_APB1ENR |= (1 << 0); // TIM2EN

    // PA0, PA1 output push-pull 2MHz
    GPIOA_CRL &= ~((0xF << 0) | (0xF << 4));
    GPIOA_CRL |= ((0x2 << 0) | (0x2 << 4));

    GPIOA_ODR |= LED1 | LED2;

    TIM2_CR1 = 0;
    TIM2_PSC = 8000 - 1; // 8M / 8000 = 1000 = 1 tick/ms
    TIM2_ARR = 50 - 1; // 50 tick = 50 ms
    TIM2_CNT = 0;

    TIM2_EGR = 1; // update PSC & ARR
    TIM2_SR = 0; // clear flag

    TIM2_DIER |= 1; // enable TIM2 update interrupt
    NVIC_ISER0 |= (1 << 28); // enable TIM2 receive interrupt

    TIM2_CR1 |= 1; // enable TIM2 counter

```

```
while (1);  
}
```

1. Khai báo thanh ghi và chức năng từng thanh ghi

- **RCC_CR** là thanh ghi điều khiển clock, dùng để bật HSI và kiểm tra HSI đã sẵn sàng.
- **RCC_CFGR** là thanh ghi cấu hình clock hệ thống, dùng để chọn HSI làm nguồn clock chính cho vi điều khiển.
- **RCC_APB2ENR** là thanh ghi cho phép clock ngoại vi trên bus APB2. GPIOA nằm trên APB2 nên set bit tương ứng trong thanh ghi này để bật clock cho GPIOA.
- **RCC_APB1ENR** là thanh ghi cho phép clock ngoại vi trên bus APB1. TIM2 nằm trên APB1 nên set bit tương ứng trong thanh ghi này để bật clock cho TIM2.
- **GPIOA_CRL** là thanh ghi cấu hình các chân PA0 đến PA7. Vì LED1 nối với PA0 và LED2 nối với PA1 nên cấu hình PA0 và PA1 ở chế độ output push-pull tốc độ 2 MHz.
- **GPIOA_ODR** là thanh ghi dữ liệu ngõ ra của GPIOA, dùng để đưa mức logic ra PA0 và PA1 điều khiển trạng thái sáng/tắt của LED1 và LED2.
- **TIM2_CR1** là thanh ghi điều khiển chính của TIM2, dùng để cấu hình chế độ hoạt động và bật tắt Timer. Trong chương trình, $TIM2_CR1 = 0$ đưa Timer về trạng thái ban đầu, sau đó set bit CEN để cho phép Timer bắt đầu đếm.
- **TIM2_DIER** là thanh ghi cho phép ngắt/DMA của TIM2. Khi bật bit UIE, TIM2 sẽ tạo ngắt mỗi khi có sự kiện cập nhật.
- **TIM2_SR** là thanh ghi trạng thái của TIM2. Trong hàm ngắt, chương trình kiểm tra bit UIF để biết Timer đã tạo sự kiện cập nhật hay chưa, sau đó xóa bit này để chuẩn bị cho lần ngắt tiếp theo.
- **TIM2_EGR** là thanh ghi tạo sự kiện của TIM2. Chương trình ghi 1 vào bit UG để tạo sự kiện cập nhật, giúp các giá trị PSC và ARR vừa cấu hình được nạp vào Timer ngay lập tức.
- **TIM2_CNT** là thanh ghi chứa giá trị đếm hiện tại của TIM2. Chương trình gán $TIM2_CNT = 0$ để đưa bộ đếm về 0 trước khi Timer bắt đầu chạy.
- **TIM2_PSC** là thanh ghi chia tần số cho TIM2. Với cấu hình clock hiện tại, TIM2 nhận clock 8 MHz, chương trình gán $TIM2_PSC = 8000 - 1$ để chia clock xuống còn 1 kHz, tức mỗi tick Timer tương ứng 1 ms.
- **TIM2_ARR** là thanh ghi chứa giá trị giới hạn đếm của Timer. Chương trình gán $TIM2_ARR = 50 - 1$, nghĩa là Timer đếm từ 0 đến 49, tổng cộng 50 tick rồi tạo Update Event (vì mỗi tick là 1 ms nên TIM2 tạo ngắt mỗi 50 ms).
- **NVIC_ISER0** là thanh ghi cho phép ngắt trong NVIC (Nested Vectored Interrupt Controller – bộ quản lý ngắt của lõi ARM). Chương trình set bit 28 để cho phép CPU nhận và xử lý ngắt từ TIM2, vì TIM2 có IRQ number là 28.

2. Logic điều khiển:

- Chương trình sử dụng TIM2 để tạo ngắt định kỳ, từ đó điều khiển LED1 và LED2 nhấp nháy theo các chu kỳ khác nhau. Đầu tiên, vi điều khiển bật nguồn clock HSI 8 MHz và chọn HSI làm clock hệ thống. Sau đó, chương trình bật clock cho GPIOA để sử dụng hai chân PA0/PA1, đồng thời bật clock cho TIM2 để Timer có thể hoạt động. PA0 và PA1 được cấu hình ở chế độ output push-pull 2MHz để điều khiển hai LED.

- TIM2 được cấu hình bằng hai thanh ghi quan trọng là PSC và ARR. Thanh ghi PSC dùng để chia tần số clock, còn ARR quy định số tick mà Timer cần đếm trước khi tạo sự kiện ngắt. Công thức tính thời gian ngắt của Timer là: $T = (PSC + 1) \times (ARR + 1) / F_Timer$

- Trong chương trình:

$$PSC = 8000 - 1$$

$$ARR = 50 - 1$$

$$F_Timer = 8.000.000 \text{ Hz}$$

$$\text{Suy ra: } T = 8000 \times 50 / 8.000.000 = 0.05 \text{ s} = 50 \text{ ms}$$

Clock ban đầu là 8 MHz, sau khi chia cho 8000 thì Timer có 1000 tick/s, nên 1 tick tương ứng 1 ms. Khi ARR được đặt là 50 - 1, Timer sẽ đếm đủ 50 tick, tức 50 ms, rồi tạo ngắt, tức TIM2 sẽ tạo ngắt sau mỗi 50 ms.

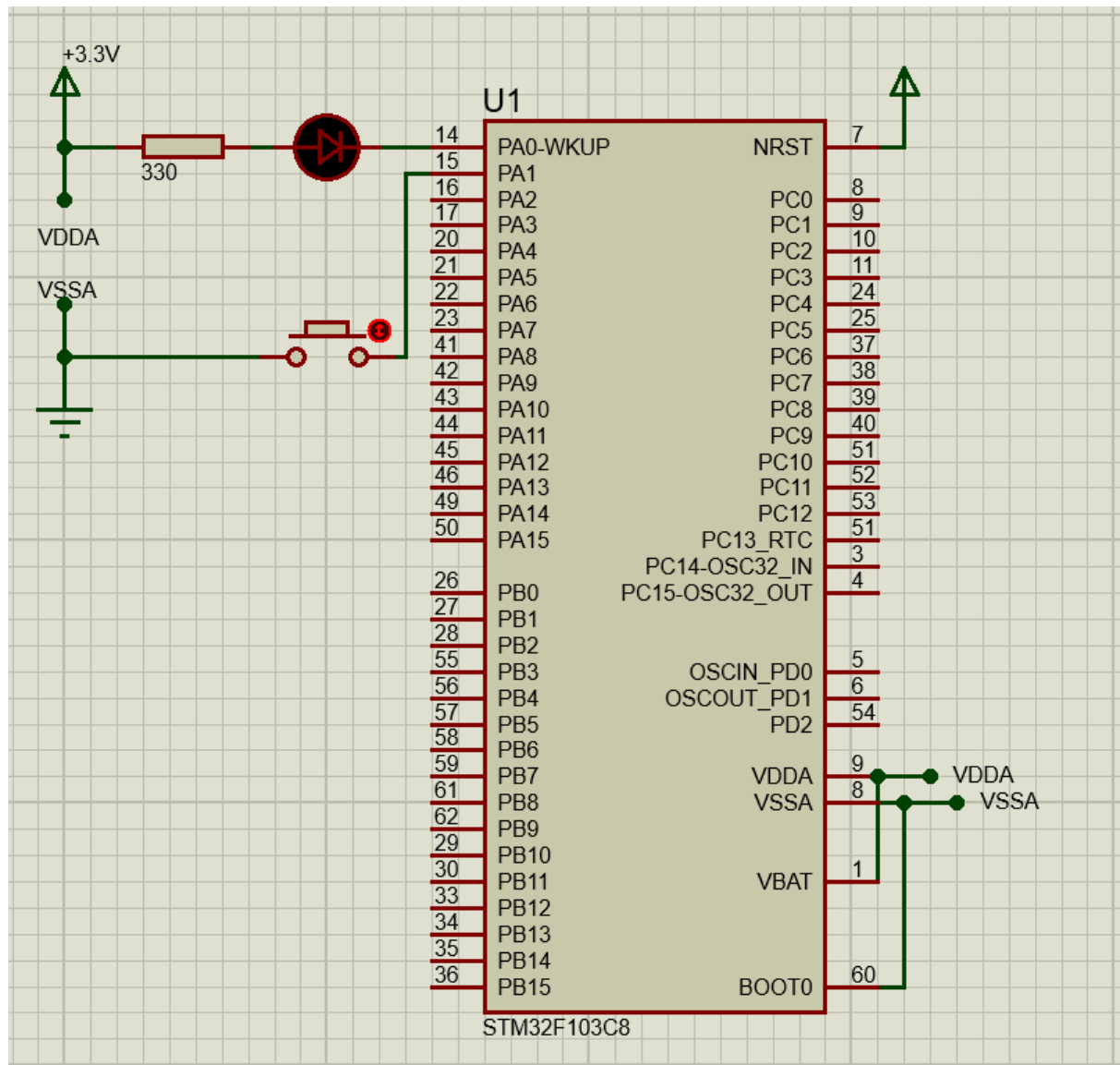
- Trong hàm TIM2_IRQHandler, chương trình kiểm tra cờ UIF để biết Timer có thật sự vừa tạo ngắt hay không. Nếu UIF bằng 1, nghĩa là Timer đã đếm đủ 50 ms. Sau đó xóa cờ UIF để xác nhận rằng ngắt hiện tại đã được xử lý, tránh việc CPU lặp lại cùng một ngắt cũ. Sau khi xóa cờ UIF, chương trình đảo trạng thái LED1 bằng lệnh: `GPIOA_ODR ^= LED1`. Lệnh này làm LED1 đổi trạng thái: nếu đang sáng thì tắt, nếu đang tắt thì sáng. Vì TIM2 tạo ngắt mỗi 50 ms, nên LED1 sẽ đảo trạng thái mỗi 50 ms. Một chu kỳ nhấp nháy đầy đủ gồm 50 ms sáng và 50 ms tắt, do đó chu kỳ nhấp nháy của LED1 là 100 ms. Đối với LED2, chương trình không đảo trạng thái ngay ở mỗi lần ngắt. Thay vào đó, biến toàn cục `count_led2` được dùng để đếm số lần ngắt đã xảy ra. Mỗi lần TIM2 tạo ngắt, biến `count_led2` tăng thêm 1. Khi `count_led2` đạt 10, nghĩa là đã có 10 lần ngắt xảy ra. Vì mỗi lần ngắt cách nhau 50 ms, nên tổng thời gian là: $10 \times 50 \text{ ms} = 500 \text{ ms}$. Khi đó chương trình reset `count_led2` về 0 và đảo trạng thái LED2. Như vậy LED2 sẽ đảo trạng thái sau mỗi 500 ms. Một chu kỳ nhấp nháy đầy đủ của LED2 gồm 500 ms sáng và 500 ms tắt, nên chu kỳ nhấp nháy của LED2 là 1 giây. Khi quan sát thực tế bằng oscilloscope, tín hiệu tại PA0 có chu kỳ khoảng 100 ms, tương ứng với LED1 nhấp nháy nhanh. Tín hiệu tại PA1 có chu kỳ khoảng 1 giây, tương ứng với LED2 nhấp nháy chậm hơn. Vì vậy, kết quả mô phỏng và kết quả đo trên oscilloscope đều đúng với yêu cầu bài tập.

II. Exercise 2

Thiết kế mạch mô phỏng gồm STM32F103C8, một nút nhấn và một LED (LED1).

Yêu cầu:

- Đếm số lần nút được nhấn.
- Khi số lần nhấn đạt 10, LED1 sẽ đảo trạng thái (toggle).
- Sau khi đạt ngưỡng, có thể tiếp tục đếm lại từ đầu.



Mạch đếm nút nhấn và đảo trạng thái

Nguyên lý hoạt động của hệ thống:

Vì điều khiển STM32F103C8 dùng PA0 để điều khiển LED và PA1 để đọc nút nhấn ở chế độ input pull-up. Khi nhấn nút, PA1 đọc mức 0, đồng thời TIM2 tạo ngắt mỗi 1 ms để chương trình kiểm tra trạng thái nút và chống dôi phím trong khoảng 20 ms.

Mỗi lần nhấn hợp lệ, biến `press_count` tăng 1. Khi đủ 10 lần nhấn, LED đảo trạng thái và `press_count` được reset về 0 để tiếp tục đếm lại.

Giải thích kết quả mô phỏng:

```
#define RCC_CR (*(volatile unsigned int*)0x40021000)
#define RCC_CFGR (*(volatile unsigned int*)0x40021004)
#define RCC_APB2ENR (*(volatile unsigned int*)0x40021018)
#define RCC_APB1ENR (*(volatile unsigned int*)0x4002101C)

#define GPIOA_CRL (*(volatile unsigned int*)0x40010800)
#define GPIOA_IDR (*(volatile unsigned int*)0x40010808)
#define GPIOA_ODR (*(volatile unsigned int*)0x4001080C)

#define TIM2_CR1 (*(volatile unsigned int*)0x40000000)
#define TIM2_DIER (*(volatile unsigned int*)0x4000000C)
#define TIM2_SR (*(volatile unsigned int*)0x40000010)
#define TIM2_EGR (*(volatile unsigned int*)0x40000014)
#define TIM2_CNT (*(volatile unsigned int*)0x40000024)
#define TIM2_PSC (*(volatile unsigned int*)0x40000028)
#define TIM2_ARR (*(volatile unsigned int*)0x4000002C)

#define NVIC_ISER0 (*(volatile unsigned int*)0xE000E100)

#define LED (1 << 0) // PA0
#define BUTTON (1 << 1) // PA1

volatile unsigned int press_count = 0;
volatile unsigned int debounce_ms = 0;

volatile unsigned char last_raw = 0;
volatile unsigned char stable_state = 0;

void TIM2_IRQHandler(void)
{
    unsigned char raw;

    if (TIM2_SR & 1) // check UIF
    {
        TIM2_SR &= ~1; // clear UIF
    }
}
```

```

raw = ((GPIOA_IDR & BUTTON) == 0); // pressed = 1

if (raw != last_raw)
{
    last_raw = raw;
    debounce_ms = 0;
}
else
{
    if (debounce_ms < 20)
    {
        debounce_ms++;
    }
    else
    {
        if (raw != stable_state)
        {
            stable_state = raw;

            if (stable_state)
            {
                press_count++;

                if (press_count >= 10)
                {
                    GPIOA_ODR ^= LED; // toggle LED
                    press_count = 0;
                }
            }
        }
    }
}
}

int main(void)
{

```



```

// HSI 8MHz
RCC_CR |= 1; // HSI ON
while (!(RCC_CR & (1 << 1))); // wait HSIRDY

RCC_CFGR &= ~(3 << 0); // SYSCLK = HSI

// Enable GPIOA, TIM2
RCC_APB2ENR |= (1 << 2); // IOPAEN
RCC_APB1ENR |= (1 << 0); // TIM2EN

// PA0 output push-pull 2MHz
GPIOA_CRL &= ~(0xF << 0);
GPIOA_CRL |= (0x2 << 0);

// PA1 input pull-up
GPIOA_CRL &= ~(0xF << 4);
GPIOA_CRL |= (0x8 << 4);

GPIOA_ODR |= LED; // LED off active-low
GPIOA_ODR |= BUTTON; // PA1 pull-up

// TIM2 interrupt 1ms
TIM2_CR1 = 0;
TIM2_PSC = 8 - 1;
TIM2_ARR = 1000 - 1;
TIM2_CNT = 0;

TIM2_EGR = 1; // update PSC & ARR
TIM2_SR = 0; // clear flag

TIM2_DIER |= 1; // enable TIM2 update interrupt
NVIC_ISER0 |= (1 << 28); // enable TIM2 receive interrupt

TIM2_CR1 |= 1; // start TIM2 counter

while (1);
}

```

1. Khai báo thanh ghi và chức năng từng thanh ghi

- **RCC_CR** là thanh ghi điều khiển clock, dùng để bật HSI và kiểm tra HSI đã sẵn sàng.
- **RCC_CFGR** là thanh ghi cấu hình clock hệ thống, dùng để chọn HSI làm nguồn clock chính cho vi điều khiển.
- **RCC_APB2ENR** là thanh ghi cho phép clock ngoại vi trên bus APB2. GPIOA nằm trên APB2 nên set bit tương ứng trong thanh ghi này để bật clock cho GPIOA.
- **RCC_APB1ENR** là thanh ghi cho phép clock ngoại vi trên bus APB1. TIM2 nằm trên APB1 nên set bit tương ứng trong thanh ghi này để bật clock cho TIM2.
- **GPIOA_CRL** là thanh ghi cấu hình các chân PA0 đến PA7. PA0 được cấu hình là output push-pull để điều khiển LED. PA1 được cấu hình ở chế độ input pull-up/pull-down, sau đó set bit PA1 trong GPIOA_ODR để chọn pull-up nội bộ.
- **GPIOA_IDR** là thanh ghi dữ liệu ngõ vào của GPIOA, dùng để đọc trạng thái nút nhấn tại chân PA1.
- **GPIOA_ODR** là thanh ghi dữ liệu ngõ ra của GPIOA. Với PA0, thanh ghi này dùng để bật/tắt LED. Với PA1 đang ở chế độ input pull-up, bit tương ứng trong GPIOA_ODR dùng để bật pull-up nội bộ.
- **TIM2_CR1** là thanh ghi điều khiển TIM2, dùng để bật hoặc tắt Timer.
- **TIM2_DIER** là thanh ghi cho phép ngắt của TIM2. Khi bật bit UIF, TIM2 sẽ tạo ngắt sau mỗi chu kỳ đã cấu hình.
- **TIM2_SR** là thanh ghi trạng thái của TIM2. Trong hàm ngắt, kiểm tra cờ UIF để biết Timer đã đếm đủ 1 ms hay chưa, sau đó xóa cờ này để chuẩn bị cho lần ngắt tiếp theo.
- **TIM2_EGR** là thanh ghi tạo sự kiện cập nhật, dùng để nạp ngay giá trị PSC và ARR mới vào Timer.
- **TIM2_CNT** là thanh ghi chứa giá trị đếm hiện tại của Timer. Chương trình gán $TIM2_CNT = 0$ để Timer bắt đầu đếm từ 0.
- **TIM2_PSC** là thanh ghi chia tần số. Trong chương trình, $TIM2_PSC = 8 - 1$ để chia clock 8 MHz xuống còn 1 MHz.
- **TIM2_ARR** là thanh ghi giới hạn đếm. Trong chương trình, $TIM2_ARR = 1000 - 1$ để Timer đếm 1000 tick rồi tạo ngắt.
- **NVIC_ISER0** là thanh ghi cho phép CPU nhận ngắt. Chương trình set bit 28 để bật ngắt TIM2 trong NVIC.

2. Logic điều khiển:

- Chương trình sử dụng TIM2 để tạo ngắt định kỳ mỗi 1 ms. Đầu tiên, vi điều khiển bật HSI 8 MHz và chọn HSI làm clock hệ thống. Sau đó, chương trình bật clock cho GPIOA để sử dụng PA0, PA1 và bật clock cho TIM2 để Timer hoạt động. PA0 được cấu hình là output

push-pull 2MHz để điều khiển LED, còn PA1 được cấu hình input pull-up/pull-down, sau đó set bit PA1 trong GPIOA_ODR để chọn pull-up nội bộ, dùng để đọc trạng thái nút nhấn.

- TIM2 được cấu hình bằng hai thanh ghi PSC và ARR. Thanh ghi PSC dùng để chia tần số clock, còn ARR quy định số tick mà Timer cần đếm trước khi tạo ngắt. Công thức tính thời gian ngắt là: $T = (PSC + 1) \times (ARR + 1) / F_Timer$

- Trong chương trình:

PSC = 8 - 1

ARR = 1000 - 1

F_Timer = 8.000.000 Hz

Suy ra: $T = 8 \times 1000 / 8.000.000 = 0.001 \text{ s} = 1 \text{ ms}$

Clock ban đầu là 8 MHz, sau khi chia cho 8 thì Timer có tần số 1 MHz, tức mỗi tick tương ứng 1 μs . Khi ARR được đặt là 1000 - 1, Timer sẽ đếm đủ 1000 tick, tức 1 ms, rồi tạo ngắt. Vì vậy, TIM2 sẽ gọi hàm TIM2_IRQHandler mỗi 1 ms.

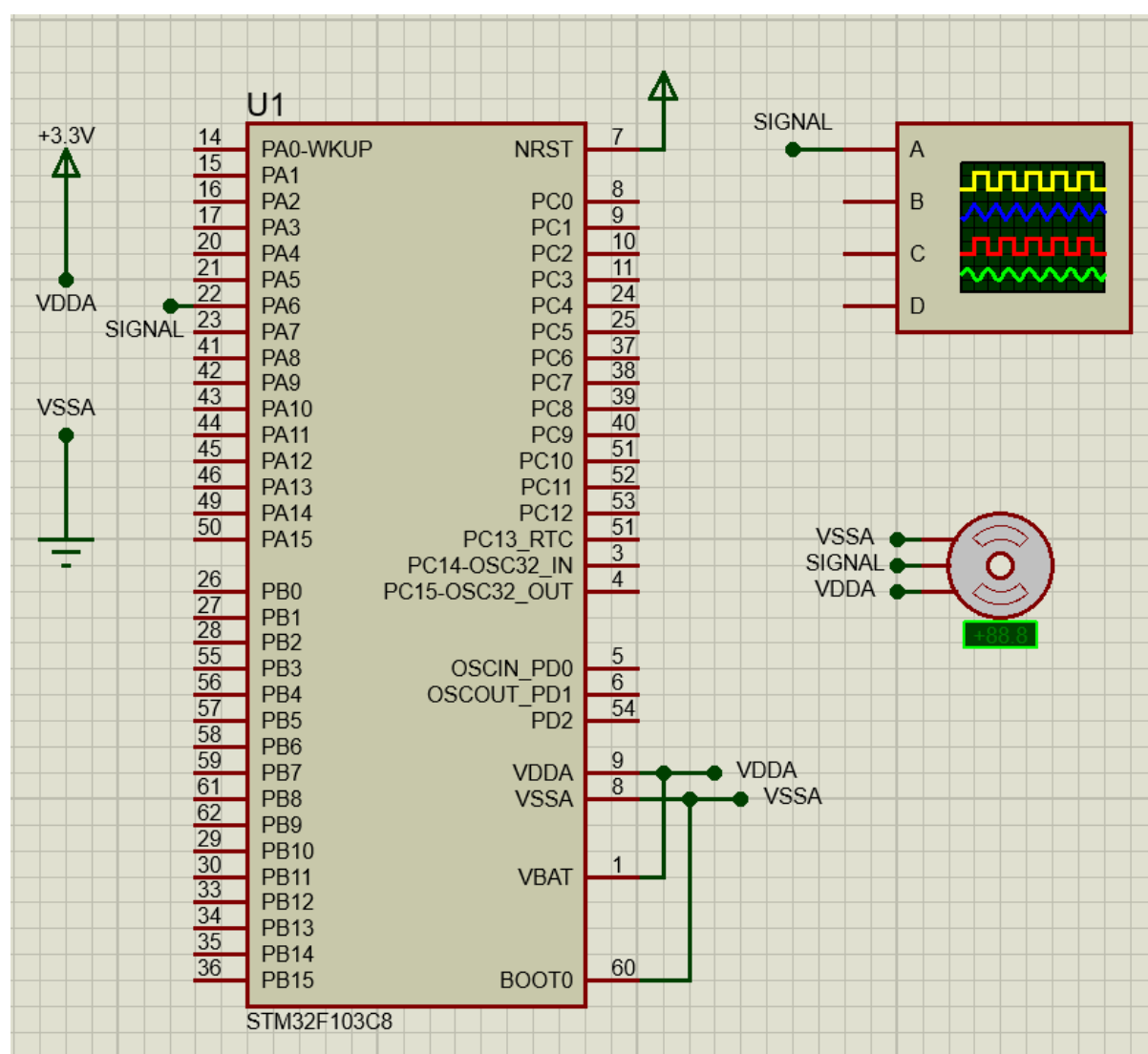
- Trong hàm TIM2_IRQHandler, chương trình kiểm tra cờ UIF để biết Timer có vừa tạo ngắt hay không. Nếu UIF bằng 1, nghĩa là Timer đã đếm đủ 1 ms. Sau đó, chương trình xóa cờ UIF để tránh xử lý lặp lại cùng một ngắt cũ. Sau khi xóa cờ UIF, chương trình đọc trạng thái nút nhấn tại PA1 bằng lệnh `raw = ((GPIOA_IDR & BUTTON) == 0)`. Vì PA1 dùng input pull-up nên khi chưa nhấn nút, PA1 ở mức 1. Khi nhấn nút, PA1 bị kéo xuống GND nên đọc được mức 0. Do đó, chương trình quy ước `raw = 1` là nút đang được nhấn.

- Để tránh hiện tượng dội phím, chương trình dùng biến `last_raw` và `debounce_ms`. Nếu trạng thái nút vừa đọc khác trạng thái trước đó, chương trình reset `debounce_ms` về 0. Nếu trạng thái nút giữ nguyên, `debounce_ms` tăng dần sau mỗi lần ngắt 1 ms. Khi `debounce_ms` đạt 20, nghĩa là trạng thái nút đã ổn định khoảng 20 ms, chương trình mới chấp nhận trạng thái đó là hợp lệ. Khi phát hiện một lần nhấn hợp lệ, biến `press_count` tăng thêm 1. Khi `press_count` đạt 10, chương trình đảo trạng thái LED bằng lệnh `GPIOA_ODR ^= LED`, sau đó reset `press_count` về 0 để tiếp tục đếm lại từ đầu. Kết quả mô phỏng là chương trình đếm số lần nhấn nút khi đủ 10 lần thì LED tại PA0 sẽ đổi trạng thái, sau đó hệ thống tiếp tục đếm lại từ đầu.

III. Exercise 3

Xây dựng mô hình mô phỏng sử dụng servo làm kim đồng hồ. Yêu cầu:

- Cứ mỗi 1 giây, servo quay một góc khoảng 6° , tương ứng với chuyển động của kim giây.
- Sau 60 giây (tương ứng một chu kỳ hoàn chỉnh hoặc đạt giới hạn góc quay 180°), servo phải quay nhanh về vị trí ban đầu (0°).
- Chu kỳ hoạt động sau đó được lặp lại liên tục.
- Sử dụng PWM từ Timer để điều khiển servo, không sử dụng delay bằng loop.



Mạch điều khiển servo làm kim đồng hồ

Nguyên lý hoạt động của hệ thống:

Vi điều khiển STM32F103C8 sử dụng chân PA6 để xuất tín hiệu PWM điều khiển servo. PA6 được cấu hình ở chế độ alternate function push-pull 2 MHz vì chân này được dùng làm ngõ ra TIM3_CH1. TIM3 tạo tín hiệu PWM có chu kỳ 20 ms, phù hợp để điều khiển servo.

Độ rộng xung PWM thay đổi từ khoảng 1000 μ s đến 2000 μ s, tương ứng với góc quay từ 0° đến 180°. TIM2 được cấu hình tạo ngắt mỗi 1 giây. Mỗi lần ngắt TIM2 xảy ra, chương trình tăng góc servo thêm 6°. Sau đó chương trình đổi giá trị góc sang độ rộng xung PWM và ghi vào TIM3_CCR1 để cập nhật vị trí servo. Khi góc đạt 180°, chương trình đưa góc quay trở về 0° và chu kỳ hoạt động được lặp lại từ đầu.

Giải thích kết quả mô phỏng:

```
#define RCC_CR (*(volatile unsigned int*)0x40021000)
#define RCC_CFGR (*(volatile unsigned int*)0x40021004)
#define RCC_APB2ENR (*(volatile unsigned int*)0x40021018)
#define RCC_APB1ENR (*(volatile unsigned int*)0x4002101C)

#define GPIOA_CRL (*(volatile unsigned int*)0x40010800)

#define TIM2_CR1 (*(volatile unsigned int*)0x40000000)
#define TIM2_DIER (*(volatile unsigned int*)0x4000000C)
#define TIM2_SR (*(volatile unsigned int*)0x40000010)
#define TIM2_EGR (*(volatile unsigned int*)0x40000014)
#define TIM2_CNT (*(volatile unsigned int*)0x40000024)
#define TIM2_PSC (*(volatile unsigned int*)0x40000028)
#define TIM2_ARR (*(volatile unsigned int*)0x4000002C)

#define TIM3_CR1 (*(volatile unsigned int*)0x40000400)
#define TIM3_CCMR1 (*(volatile unsigned int*)0x40000418)
#define TIM3_CCER (*(volatile unsigned int*)0x40000420)
#define TIM3_EGR (*(volatile unsigned int*)0x40000414)
#define TIM3_SR (*(volatile unsigned int*)0x40000410)
#define TIM3_CNT (*(volatile unsigned int*)0x40000424)
#define TIM3_PSC (*(volatile unsigned int*)0x40000428)
#define TIM3_ARR (*(volatile unsigned int*)0x4000042C)
#define TIM3_CCR1 (*(volatile unsigned int*)0x40000434)

#define NVIC_ISER0 (*(volatile unsigned int*)0xE000E100)

volatile unsigned int angle = 0;

void TIM2_IRQHandler(void)
{
    unsigned int pulse;
```

```

if (TIM2_SR & 1) // check UIF
{
    TIM2_SR &= ~1; // clear UIF

    if (angle >= 180)
        angle = 0;
    else
    {
        angle += 6;
        if (angle > 180)
            angle = 180;
    }

    // angle to pulse 1000µs - 2000µs
    pulse = 1000 + (angle * 1000 + 90) / 180;

    TIM3_CCR1 = pulse; // update PWM duty
}
}

int main(void)
{
    // HSI 8MHz
    RCC_CR |= 1; // HSI ON
    while (!(RCC_CR & (1 << 1))); // wait HSIRDY

    RCC_CFGR &= ~(3 << 0); // SYSCLK = HSI

    // Enable GPIOA, AFIO
    RCC_APB2ENR |= (1 << 0); // AFIOEN
    RCC_APB2ENR |= (1 << 2); // IOPAEN

    // PA6 alternate function push-pull 2MHz
    GPIOA_CRL &= ~(0xF << 24);
    GPIOA_CRL |= (0xA << 24);

    // Enable TIM3

```

```

RCC_APB1ENR |= (1 << 1); // TIM3EN

// TIM3 PWM 20ms
TIM3_CR1 = 0;
TIM3_CCER = 0;

TIM3_PSC = 8 - 1; // 8MHz / 8 = 1MHz = 1 tick/μs
TIM3_ARR = 20000 - 1; // 20000 tick = 20ms
TIM3_CNT = 0;

TIM3_CCMR1 &= ~(0xFF);
TIM3_CCMR1 |= (6 << 4); // PWM mode 1
TIM3_CCMR1 |= (1 << 3); // enable preload

TIM3_CCER |= (1 << 0); // enable CH1 output
TIM3_CR1 |= (1 << 7); // enable auto-reload preload

TIM3_CCR1 = 1000; // initial pulse 1000μs

TIM3_EGR = 1; // update PSC & ARR
TIM3_SR = 0; // clear flag
TIM3_CR1 |= 1; // start TIM3 counter

// Enable TIM2
RCC_APB1ENR |= (1 << 0); // TIM2EN

// TIM2 interrupt 1s
TIM2_CR1 = 0;
TIM2_DIER = 0;

TIM2_PSC = 8000 - 1; // 8MHz / 8000 = 1000Hz = 1 tick/ms
TIM2_ARR = 1000 - 1; // 1000 tick = 1s
TIM2_CNT = 0;

TIM2_EGR = 1; // update PSC & ARR
TIM2_SR = 0; // clear flag

TIM2_DIER |= 1; // enable TIM2 update interrupt

```

```

NVIC_ISER0 |= (1 << 28); // enable TIM2 receive interrupt

TIM2_CR1 |= 1; // start TIM2 counter

while (1);
}

```

1. Khai báo thanh ghi và chức năng từng thanh ghi

- **RCC_CR** là thanh ghi điều khiển clock, dùng để bật HSI và kiểm tra HSI đã sẵn sàng.
- **RCC_CFGR** là thanh ghi cấu hình clock hệ thống, dùng để chọn HSI làm nguồn clock chính cho vi điều khiển.
- **RCC_APB2ENR** là thanh ghi cho phép clock ngoại vi trên bus APB2. GPIOA nằm trên APB2 nên set bit tương ứng trong thanh ghi này để bật clock cho GPIOA và AFIO.
- **RCC_APB1ENR** là thanh ghi cho phép clock ngoại vi trên bus APB1. TIM2 và TIM3 nằm trên APB1 nên set bit tương ứng để bật clock cho TIM2 và TIM3.
- **GPIOA_CRL** là thanh ghi cấu hình các chân PA0 đến PA7. PA6 được cấu hình ở chế độ alternate function push-pull 2 MHz để xuất tín hiệu PWM từ TIM3_CH1 điều khiển servo.
- **TIM2_CR1** là thanh ghi điều khiển chính của TIM2, dùng để cấu hình chế độ hoạt động và bật tắt Timer. Trong chương trình, TIM2 được dùng để tạo ngắt định kỳ mỗi 1 giây.
- **TIM2_DIER** là thanh ghi cho phép ngắt/DMA của TIM2. Khi bật bit UIE, TIM2 sẽ tạo ngắt mỗi khi có sự kiện cập nhật.
- **TIM2_SR** là thanh ghi trạng thái của TIM2. Trong hàm ngắt, chương trình kiểm tra bit UIF để biết Timer đã tràn hay chưa, sau đó xóa bit này để chuẩn bị cho lần ngắt tiếp theo.
- **TIM2_EGR** là thanh ghi tạo sự kiện của TIM2. Chương trình ghi 1 vào bit UG để tạo Update Event, giúp các giá trị PSC và ARR vừa cấu hình được nạp vào Timer ngay lập tức.
- **TIM2_CNT** là thanh ghi chứa giá trị đếm hiện tại của TIM2. Chương trình gán TIM2_CNT = 0 để đưa bộ đếm về 0 trước khi Timer bắt đầu chạy.
- **TIM2_PSC** là thanh ghi chia tần số cho TIM2. Với cấu hình clock hiện tại, TIM2 nhận clock 8 MHz, chương trình gán TIM2_PSC = 8000 - 1 để chia clock xuống còn 1 kHz, tức mỗi tick Timer tương ứng 1 ms.
- **TIM2_ARR** là thanh ghi chứa giá trị giới hạn đếm của Timer. Chương trình gán TIM2_ARR = 1000 - 1, nghĩa là TIM2 đếm 1000 tick, tương ứng 1 giây, rồi tạo ngắt.
- **TIM3_CR1** là thanh ghi điều khiển chính của TIM3, cấu hình chế độ hoạt động và bật tắt Timer với TIM3 được dùng để tạo tín hiệu PWM điều khiển servo.
- **TIM3_CCMR1** là thanh ghi cấu hình chế độ Capture/Compare cho kênh 1 của TIM3. Chương trình cấu hình TIM3_CH1 ở chế độ PWM mode 1.

- **TIM3_CCER** là thanh ghi cho phép ngõ ra Capture/Compare. Chương trình set bit CC1E để cho phép TIM3_CH1 xuất tín hiệu PWM ra chân PA6.
- **TIM3_EGR** là thanh ghi tạo sự kiện của TIM3, dùng để nạp ngay giá trị PSC và ARR mới vào Timer.
- **TIM3_SR** là thanh ghi trạng thái của TIM3, dùng để chứa các cờ trạng thái của Timer. Trong chương trình, thanh ghi này được xóa trước khi TIM3 bắt đầu chạy.
- **TIM3_CNT** là thanh ghi chứa giá trị đếm hiện tại của TIM3. Chương trình gán $TIM3_CNT = 0$ để đưa bộ đếm về 0 trước khi Timer bắt đầu chạy.
- **TIM3_PSC** là thanh ghi chia tần số cho TIM3. Với cấu hình clock hiện tại, TIM3 nhận clock 8 MHz, chương trình gán $TIM3_PSC = 8 - 1$ để chia clock xuống còn 1 MHz, tức mỗi tick Timer tương ứng 1 μs .
- **TIM3_ARR** là thanh ghi giới hạn đếm của TIM3. Trong chương trình, $TIM3_ARR = 20000 - 1$, nghĩa là TIM3 đếm 20000 tick, tương ứng 20 ms, rồi lặp lại chu kỳ PWM.
- **TIM3_CCR1** là thanh ghi Capture/Compare kênh 1 của TIM3. Trong chế độ PWM, giá trị TIM3_CCR1 quyết định độ rộng mức HIGH của xung PWM. Giá trị này thay đổi từ khoảng 1000 đến 2000, tương ứng với xung 1000 μs đến 2000 μs để điều khiển góc servo.
- **NVIC_ISER0** là thanh ghi cho phép ngắt trong NVIC (Nested Vectored Interrupt Controller – bộ quản lý ngắt của lõi ARM). Chương trình set bit 28 để cho phép CPU nhận và xử lý ngắt từ TIM2, vì TIM2 có IRQ number là 28.

2. Logic điều khiển:

- Chương trình sử dụng TIM3 để tạo tín hiệu PWM điều khiển servo và sử dụng TIM2 để tạo ngắt định kỳ mỗi 1 giây. Đầu tiên, vi điều khiển bật HSI 8 MHz và chọn HSI làm clock hệ thống. Sau đó, chương trình bật clock cho GPIOA, AFIO, TIM3 và TIM2. PA6 được cấu hình ở chế độ alternate function push-pull 2 MHz vì PA6 là chân TIM3_CH1, dùng để xuất PWM từ Timer ra servo. TIM3 được cấu hình để tạo PWM có chu kỳ 20 ms. Công thức tính chu kỳ Timer là: $T = (PSC + 1) \times (ARR + 1) / F_Timer$

- Trong chương trình:

$$PSC = 8 - 1$$

$$ARR = 20000 - 1$$

$$F_Timer = 8.000.000 \text{ Hz}$$

$$\text{Suy ra: } T = 8 \times 20000 / 8.000.000 = 0.02 \text{ s} = 20 \text{ ms}$$

Clock ban đầu là 8 MHz, sau khi chia cho 8 thì TIM3 có 1000000 tick/s, tức mỗi tick tương ứng 1 μs . Khi ARR được đặt là 20000 - 1, TIM3 sẽ đếm đủ 20000 tick, tức 20 ms, rồi lặp lại chu kỳ PWM. Giá trị TIM3_CCR1 quyết định độ rộng xung HIGH của PWM. Ban đầu, $TIM3_CCR1 = 1000$, nghĩa là xung HIGH rộng 1000 μs , tương ứng gần vị trí 0° của servo. Khi $TIM3_CCR1 = 2000$, xung HIGH rộng 2000 μs , tương ứng gần vị trí 180° của servo.

- TIM2 được cấu hình để tạo ngắt mỗi 1 giây. Trong chương trình:

$$PSC = 8000 - 1$$

$$ARR = 1000 - 1$$

$$F_Timer = 8.000.000 \text{ Hz}$$

$$\text{Suy ra: } T = 8000 \times 1000 / 8.000.000 = 1 \text{ s}$$

Clock ban đầu là 8 MHz, sau khi chia cho 8000 thì TIM2 có 1000 tick/s, tức mỗi tick tương ứng 1 ms. Khi ARR được đặt là 1000 - 1, TIM2 sẽ đếm đủ 1000 tick (1s), rồi tạo ngắt. Trong hàm TIM2_IRQHandler, chương trình kiểm tra cờ UIF để biết TIM2 có vừa tạo ngắt hay không. Nếu UIF bằng 1, nghĩa là TIM2 đã đếm đủ 1 giây. Sau đó chương trình xóa cờ UIF để tránh xử lý lặp lại cùng một ngắt cũ.

- Sau mỗi lần ngắt TIM2, biến angle tăng thêm 6°. Nếu angle đạt 180° thì chương trình đưa angle về 0 để servo quay lại vị trí ban đầu. Sau đó, chương trình đổi góc angle thành độ rộng xung PWM bằng công thức: $\text{pulse} = 1000 + (\text{angle} \times 1000 + 90) / 180$. Giá trị pulse này được ghi vào TIM3_CCR1 để cập nhật độ rộng xung PWM. Kết quả mô phỏng là servo quay từng bước giống kim giây đồng hồ. Cứ mỗi 1 giây, servo tăng khoảng 6°. Khi đạt 180°, servo quay về 0° và tiếp tục lặp lại. Trên oscilloscope, tín hiệu PWM tại PA6 có chu kỳ 20 ms, còn độ rộng xung HIGH thay đổi từ khoảng 1000 μs đến 2000 μs theo góc quay của servo.

IV. References

- TS. Vũ Đức Lung, TS. Lê Quang Minh, ThS. Phan Đình Duy, *Giáo trình Vi điều khiển*, Trường Đại học Công nghệ Thông tin - ĐHQG TP.HCM, 2015
- STM32F101xx, STM32F102xx, STM32F103xx, STM32F105xx and STM32F107xx advanced Arm®-based 32-bit MCUs, *RM0008 Reference manual*, 2021
- Link video:
 - Bài 1:
<https://drive.google.com/file/d/1X7CWhbkjHxM2rNB0h8t0GquQZpO1CsEo/view?usp=sharing>
 - Bài 2:
https://drive.google.com/file/d/1xKSvGV4h0jZCkFXTNK7RBrBJAXLwFVk_/view?usp=sharing
 - Bài 3:
<https://drive.google.com/file/d/1VTdRVgg44dkuPP7W9QaKs3a4xdQxWUK8/view?usp=sharing>